# Mobile Developer Guide — Morales

Prepared for external developer handoff · Morales Medical Travel Safety

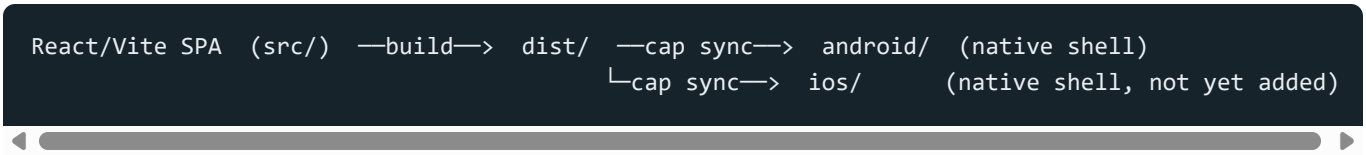
For the developer picking up the iOS/Android build. This is an orientation doc, not a tutorial — it tells you what exists, what doesn't, and the two or three things that will bite you if you don't know them going in.

## What this app is

Morales is a medical-travel concierge platform: clients book dental/aesthetic procedures with verified doctors abroad, and the platform coordinates travel logistics, companions, security escorts, and post-operative recovery monitoring. A hard safety layer sits underneath all of it — see "The one rule that matters" below before you touch anything.

## Architecture — read this before opening Android Studio or Xcode

**There is no separate mobile codebase.** The mobile apps are Capacitor wrapping the same React 18 + Vite single-page app that runs on the web, in a native WebView. One React codebase, three targets (web, iOS, Android). There is currently **zero Capacitor plugin code in** `src/` — no native camera/push/biometric integration yet. If you fix a bug in `src/`, you've fixed it on all three platforms at once. If a feature needs a real native capability (push notifications, biometric unlock, native camera), that's new work, not a bug fix — see "What's not built yet."




Config lives in `capacitor.config.ts` at the repo root — read the comments in that file, they explain *why* each setting is what it is (dark splash background so the SOS path never flashes white, HTTPS-only scheme so Web Crypto works, mixed content blocked so a hostile network can't rewrite safety traffic). Don't change those settings without reading why they're set first.

## Current state

| Platform | Status |
|---|---|
| Web | Live, deployed via Base44 (see root `CLAUDE.md` for the deploy model) |
| Android | Native project scaffolded and **committed** at `android/` (54 tracked files — manifest, Gradle config, launcher icons, splash screens; build output is gitignored). `applicationId` is `com.morales.medicaltravel`, `minSdk` 24, `targetSdk` 36, `versionCode` 1 / `versionName` "1.0" — never built for a store yet. |
| iOS | **Not added to the repo.** `cap add ios` requires CocoaPods + Xcode, i.e. a real Mac. Whoever has one runs `npm run mobile:add:ios` once and commits the result. |

Mobile-specific WebView bugs that were already found and fixed — know these exist so you don't reintroduce them:

- Fixed nav ignoring `env(safe-area-inset-*)` → rendered under the iOS status bar.
- An inline `style={{display:'flex'}}` next to `className="hidden lg:flex"` — inline style always wins, so the `hidden` class did nothing and the desktop panel rendered (and scrolled sideways) on phones. Watch for this pattern generally.
- iOS sizes `vh` against the large viewport (hides content behind the URL bar) — use `dvh` where it matters.
- Touch inputs need a 16px floor or iOS auto-zooms on focus.
- `navigator.geolocation` inside an Android WebView silently reports "denied" without the manifest permissions — already declared in `AndroidManifest.xml` (`ACCESS_FINE_LOCATION` / `ACCESS_COARSE_LOCATION`, GPS hardware marked *not required* since non-travelers use the app without ever needing location).

## The one rule that matters — read before touching safety-adjacent code

> *"We built this to save lives, not to make profit."*

If a client picks a combination of procedures that's clinically dangerous together, the app refuses — **RED is a hard block, no bypass, no exception**. That decision is computed by pure deterministic code (`base44/functions/_shared/safeTEngine.ts`), never by the AI. The AI is only ever allowed to narrate a decision already written by deterministic code, or add caution — never clear it, never approve, never override. A CI red-team suite (`tests/redteam/safety-redteam.spec.js`) fails the build if this is ever weakened. Full detail is in the root `CLAUDE.md` under "The M Principle" and "Safety-Decision Architecture" — read it once, it explains invariants you must not regress, on any platform, including mobile.

Practically: nothing about wrapping this in Capacitor should ever route a safety decision through anything but that same deterministic path. The mobile app talks to the exact same Base44 backend as the web app — there is no mobile-only API, no client-side safety logic to "port."

## Building and running

```
npm install
npm run dev              # web dev server, hot reload — do most of your work here
npm run mobile:sync      # vite build + cap sync -- copies dist/ into android/ (and ios/ once adde
npm run mobile:open:android   # opens the native project in Android Studio
npm run mobile:open:ios       # (once ios/ exists) opens it in Xcode
```

There's no mobile-only dev loop — you build the web bundle and Capacitor copies it in. Iterate in the browser first (fast), sync and check the native shell periodically (slow, needs a device/emulator) rather than round-tripping through Android Studio for every change.

**Auth won't complete on `localhost`.** This checkout has no Base44 session credentials — sign-in flows need to be verified against the deployed app. See root `CLAUDE.md` → "Deploying & Environment" for `.env.local` setup and the Base44 publish model.

## Testing

```
npm run lint && npm run typecheck && npm test   # before any commit
npx playwright test --project=redteam           # safety invariants — must stay green, no network
```

Playwright's `public` / `authenticated` projects run against the **deployed** app, not local — local dev can't complete a Base44 session. There's currently no device-lab / real-hardware test pass; verification of native behavior (permissions prompts, splash timing, safe-area insets) has been manual, on real devices, not automated. If you add native plugin code, you're also adding the first gap in that coverage — plan for manual verification on both a real iPhone and a real Android device, not just emulators (WebKit vs Chromium differences already caused real bugs here).

## What's not built yet (don't assume it exists)

- No native plugins at all — no push notifications, no native camera/biometric/contacts integration. Anything requiring those is new scope.
- No iOS project committed (see above).

- No app has ever been submitted to the Play Store or App Store — no store listing, screenshots, signing config, or release keystore exists yet.
- `versionCode` / `versionName` are still the scaffold defaults (`1` / `"1.0"`) — nobody has wired up a real release-versioning process.

## Where to go next

- Root `CLAUDE.md` — full architecture: Base44 SDK, auth flow, routing, edge functions, RBAC, security-critical systems (PIN vault, audit hash chain). Required reading before backend-adjacent work.
- `docs/DEMO_RUNBOOK.md` — what the app currently demonstrates end-to-end, and what's explicitly not ready to show live (payments, final booking submission).
- `capacitor.config.ts` — inline comments explain every mobile-specific setting.

---